

A four-page live analytics dashboard for a fictional ride-sharing company called **SwiftRide**, with a machine learning fare prediction model built in. Every number comes from a SQLite database.

Page 1 – Executive Overview	KPIs, revenue trend, city map
Page 2 – Trip Analytics	Demand by hour, vehicle breakdown
Page 3 – Driver Performance	Rankings, ratings, earnings
Page 4 – ML Fare Predictor	Train a model, predict live fares

[illegible]

SQLite is a database that lives entirely inside a single file on your computer. No server, no installation, no configuration. It speaks the same SQL you have been writing all semester – SELECT, JOIN, GROUP BY, all of it. In production you would swap it for PostgreSQL and change almost nothing else. For a demo it is perfect because it just works.

pandas is what brings database results into Python. When you run a SQL query, pandas turns the rows and columns into a DataFrame — the same structure you use in data science for everything. Once data is a DataFrame, every Python tool can work with it.

plotly turns DataFrames into interactive charts. Bar charts, line charts, scatter plots, heatmaps — all rendered in the browser, all zoomable and hoverable. It takes about one line of code per chart once your data is shaped correctly.

Streamlit is the thing that turns a plain Python script into a web application. Normally building a web app means learning HTML, CSS, JavaScript, and a backend framework. Streamlit removes all of that. You write Python, add a few `st.` calls, run one command, and a browser tab opens with your app. It hot-reloads as you save, which means you see changes instantly.

scikit-learn is Python's standard machine learning library. It gives you algorithms — Random Forest, Linear Regression, K-Means, and dozens more — each with the same two-line interface: `.fit()` to train, `.predict()` to use. You do not implement the math. You describe what you want, feed it data, and it learns.

I am building a demo analytics dashboard for a Pakistani ride-sharing company called SwiftRide. I need you to write a complete Python script called generate_data.py that creates a SQLite database file called swiftride.db with realistic fake data.

DATABASE SCHEMA – create exactly these 6 tables:

1. cities (city_id, city_name, province, population, is_active)
Data: Karachi, Lahore, Islamabad, Rawalpindi, Peshawar, Quetta, Multan, Faisalabad – all Pakistani cities with real provinces.
2. drivers (driver_id, name, phone, city_id, vehicle_type, vehicle_model, rating, total_trips, is_active, joined_date)
 - 150 drivers distributed across the 8 cities
 - vehicle_type must be one of: Bike, Rickshaw, Car, SUV
 - Bikes and Rickshaws are cheaper vehicle types
 - rating between 3.5 and 5.0, realistic distribution
 - joined_date between 2022-01-01 and 2024-06-01
 - 85% of drivers are active
 - Use realistic Pakistani male and female names
3. riders (rider_id, name, phone, email, city_id, signup_date, total_trips, rating)
 - 800 riders distributed across cities
 - signup_date between 2022-06-01 and 2024-12-01
 - rating between 3.0 and 5.0
 - Use realistic Pakistani names
4. trips (trip_id, rider_id, driver_id, city_id, pickup_area, dropoff_area, vehicle_type, distance_km, duration_mins, fare_pkr, trip_date, trip_hour, day_of_week, status, is_raining, is_peak_hour, surge_multiplier)
 - 7000 trips between 2023-01-01 and 2024-12-31
 - status: 92% completed, 5% cancelled, 3% no_show
 - trip_hour between 0 and 23, peak hours are 7-9 and 17-20
 - is_peak_hour = 1 if trip_hour in peak hours else 0
 - is_raining = 1 for about 15% of trips randomly
 - distance_km: Bike 1-8km, Rickshaw 1-6km, Car 2-20km, SUV 3-25km
 - duration_mins: roughly distance * 4 minutes + random noise
 - Base fare logic (in PKR):
 - Bike: $50 + (\text{distance} * 25)$
 - Rickshaw: $60 + (\text{distance} * 30)$
 - Car: $100 + (\text{distance} * 45)$
 - SUV: $150 + (\text{distance} * 60)$
 - surge_multiplier: 1.0 normally, 1.3 during peak, 1.5 during rain+peak
 - fare_pkr = base_fare * surge_multiplier + random noise (+/- 20 PKR)
 - Round fare_pkr to nearest 10
 - day_of_week: 0=Monday to 6=Sunday
 - pickup_area and dropoff_area: use realistic area names for each city (e.g. for Karachi: DHA, Gulshan, Clifton, Saddar, Korangi, for Lahore: Gulberg, DHA, Model Town, Johar Town, etc.)
 - Only assign trips to active drivers in the matching city
5. payments (payment_id, trip_id, amount_pkr, payment_method, payment_status, paid_at)
 - One payment per completed trip
 - payment_method: 60% cash, 25% card, 15% wallet
 - payment_status: 98% completed, 2% failed
 - paid_at: same date as trip_date

- Only 65% of completed trips get a review
- rider_rating_given: rating rider gives to driver (1-5)
- driver_rating_given: rating driver gives to rider (1-5)
- comment_sentiment: positive, neutral, or negative
(70% positive, 20% neutral, 10% negative)

- Use only Python standard library + pandas + numpy + sqlite3
- Use numpy random seed 42 for reproducibility
- Print progress as it runs: "Creating cities...", "Creating drivers..." etc.
- At the end print a summary: row count for each table
- The script must run completely without errors
- Use proper foreign key relationships in the INSERT logic (e.g. trips must reference existing driver and rider IDs)
- Make seasonal patterns realistic: more trips in winter evenings, fewer in summer afternoons
- Distribute trips unevenly across cities: Karachi and Lahore get 35% of trips combined, smaller cities get less

[illegible]

I have a SQLite database called swiftride.db in my current directory. It has these tables: cities, drivers, riders, trips, payments, reviews. The trips table has columns: trip_id, rider_id, driver_id, city_id, pickup_area, dropoff_area, vehicle_type, distance_km, duration_mins, fare_pkr, trip_date, trip_hour, day_of_week, status, is_raining, is_peak_hour, surge_multiplier.

Build me a complete Streamlit application in a single file called app.py.

SETUP REQUIREMENTS:

- st.set_page_config: title="SwiftRide Analytics", page_icon="🚗", layout="wide"
- Sidebar with st.sidebar.radio navigation between 4 pages:
"Executive Overview", "Trip Analytics",
"Driver Performance", "ML Fare Predictor"
- A database helper function using sqlite3 that takes a SQL string and returns a pandas DataFrame. Decorate it with @st.cache_data(ttl=60)
- A clean sidebar footer showing "SwiftRide Analytics v1.0" and "Powered by Claude + Streamlit"

PAGE 1: EXECUTIVE OVERVIEW

Section A – Five KPI cards in one row:

- Total Revenue (PKR): sum of fare_pkr for completed trips, formatted as PKR X,XXX,XXX
- Total Trips: count of all trips
- Completed Trips: count where status = completed
- Completion Rate: completed / total as percentage
- Average Fare (PKR): avg fare_pkr for completed trips

Section B – Monthly revenue trend (line chart, plotly express):

- X axis: month (format trip_date as YYYY-MM)
- Y axis: total fare_pkr
- Title: "Monthly Revenue Trend"
- Add a vertical dashed line annotation at the most recent month

Section C – Two charts side by side:

- Left: Bar chart – total completed trips by city_name (join trips to cities on city_id), sorted descending
Title: "Trips by City"
- Right: Pie chart – trip share by vehicle_type for completed trips
Title: "Fleet Mix"

Section D – Key metrics table at bottom:

- Show city-level summary: city_name, total_trips, total_revenue, avg_fare, completion_rate_pct
- Format revenue with comma separators

PAGE 2: TRIP ANALYTICS

Section A – Heatmap: trips by hour of day (X) vs day of week (Y)

- Use plotly go.Heatmap.
- Y axis labels: Mon, Tue, Wed, Thu, Fri, Sat, Sun
- X axis: 0 to 23 (hours)
- Colour scale: Blues
- Title: "Trip Demand Heatmap (Hour vs Day)"

Section B – Two charts side by side:

- Left: Bar chart comparing average fare by vehicle_type
- Right: Scatter plot: distance_km (X) vs fare_pkr (Y), coloured by vehicle_type, for a random sample of 500 trips
Title: "Fare vs Distance"

Section C – Peak vs Off-Peak analysis:

Bar chart with two bars per vehicle type:

average fare during peak hours vs average fare off-peak

Title: "Peak vs Off-Peak Fares by Vehicle Type"

Section D – Rain impact box:

Use st.metric to show:

- Average fare when raining vs not raining

- Average trips per day when raining vs not raining

Add st.info explaining what surge pricing does during rain

PAGE 3: DRIVER PERFORMANCE

Section A – Top 10 drivers leaderboard:

Query: driver name, city, vehicle_type, total completed trips, total earnings (sum of fare_pkr), average rating from reviews

(rider_rating_given), completion rate

Join drivers to trips to reviews

Sort by total_earnings descending

Display as a styled dataframe

Section B – Two charts side by side:

Left: Bar chart of average driver rating by city

Right: Bar chart of average earnings per trip by vehicle_type

Section C – Driver activity over time:

Line chart showing number of unique active drivers per month (drivers who completed at least one trip that month)

Title: "Active Drivers Per Month"

Section D – Rating distribution:

Histogram of rider_rating_given from reviews

Title: "Distribution of Driver Ratings"

PAGE 4: ML FARE PREDICTOR

This page trains a Random Forest model and lets users predict fares.

MODEL TRAINING (runs once, cached with @st.cache_resource):

Features: distance_km, duration_mins, is_peak_hour, is_raining, surge_multiplier, day_of_week, trip_hour

Plus one-hot encoded: vehicle_type

Target: fare_pkr

Use only completed trips for training

Split 80/20 train/test with random_state=42

Train RandomForestRegressor(n_estimators=100, random_state=42)

Calculate: R^2 score, MAE, RMSE on test set

DISPLAY:

Section A – Model performance metrics in three st.metric cards:

R^2 Score, MAE (PKR), RMSE (PKR)

Add an st.success if $R^2 > 0.85$, st.warning if lower

Section B – Feature importance bar chart:

Horizontal bar chart of feature importances from the model

Sort by importance descending

Title: "What Drives Fare Prices?"

Section C – Live Fare Predictor (the interactive part):

```
st.subheader("Predict a Fare")
```

Input widgets:

- vehicle_type: st.selectbox with options Bike/Rickshaw/Car/SUV
- distance_km: st.slider 1.0 to 30.0, step 0.5
- trip_hour: st.slider 0 to 23
- day_of_week: st.selectbox Monday through Sunday (maps to 0-6)
- is_raining: st.checkbox "Currently raining?"

Calculate is_peak_hour and surge_multiplier automatically from the inputs (same logic as the data generation).

Estimate duration_mins as distance_km * 4.

When user changes any input, show the predicted fare in a large st.metric card with PKR prefix.

Add a comparison: show the average actual fare for that vehicle type from the database, and whether the prediction is above or below average.

Section D – Actual vs Predicted scatter plot:

On test set, plot actual fare (X) vs predicted fare (Y)

Add a diagonal reference line showing perfect prediction

Title: "Actual vs Predicted Fares"

GENERAL REQUIREMENTS:

- All SQL queries use the sqlite3 helper function
- Database file path is "swiftride.db" (same directory as app.py)
- All plotly charts use plotly.express or plotly.graph_objects
- Charts have proper titles, axis labels, and colour schemes
- Use st.spinner("Loading...") around any heavy computation
- The entire app must run without errors from top to bottom
- Do not use any external CSS or JavaScript

Write the complete app.py. Do not truncate any section.